EPL and Esper

The DSMS-like part of the language

POLITECNICO
MILANO 1863

Emanuele Della Valle

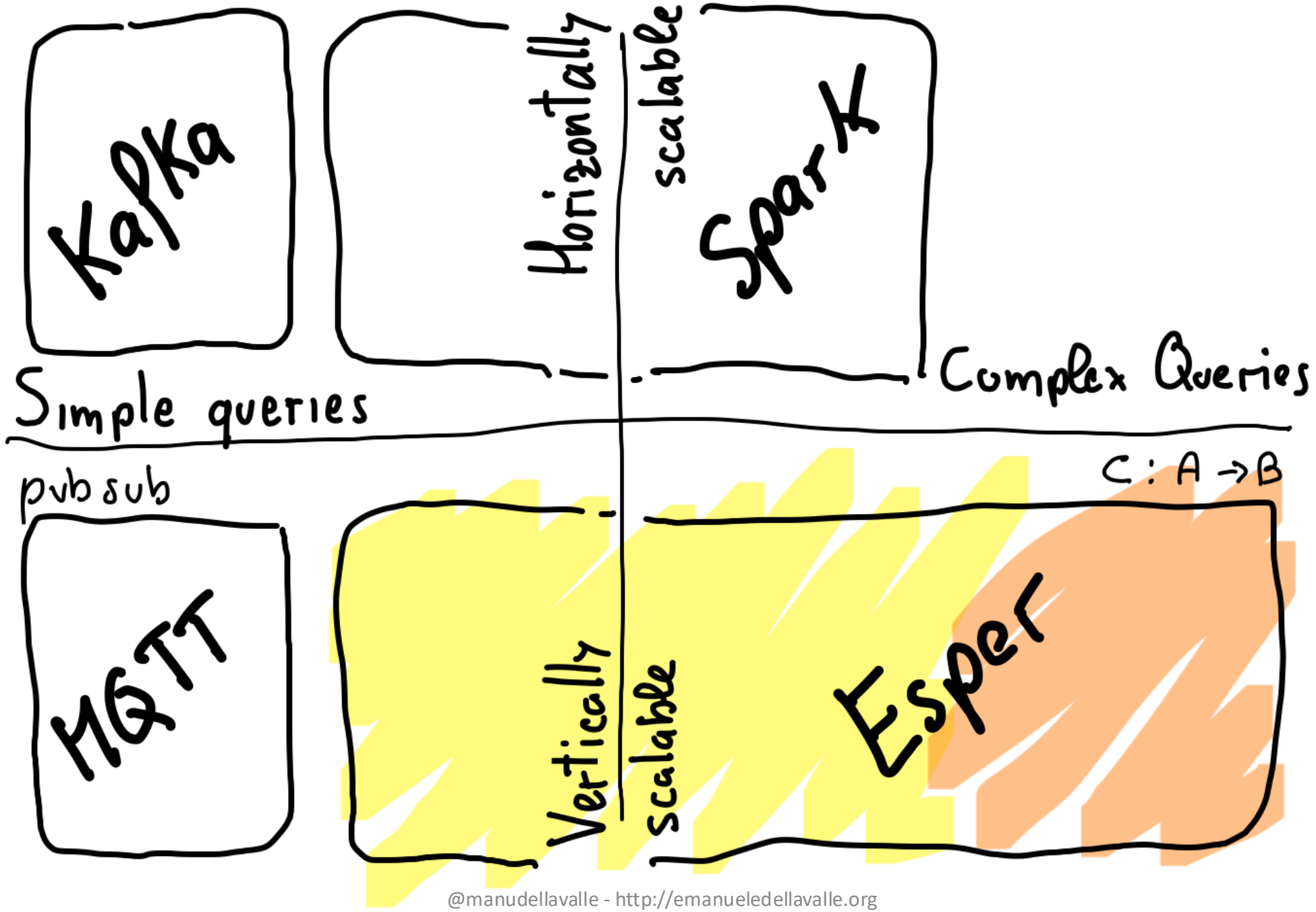
prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml

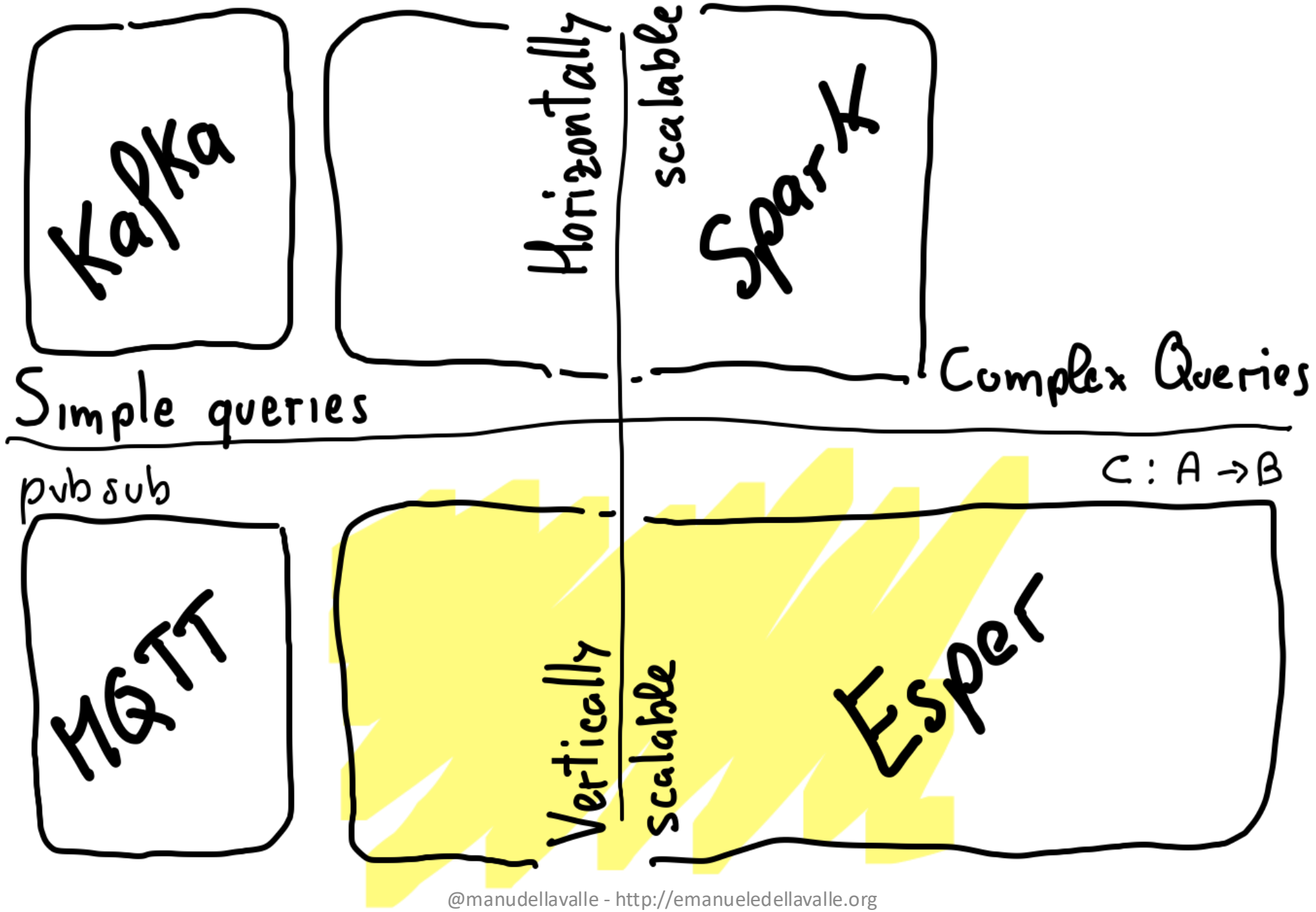
POLITECNICO
MILANO 1863

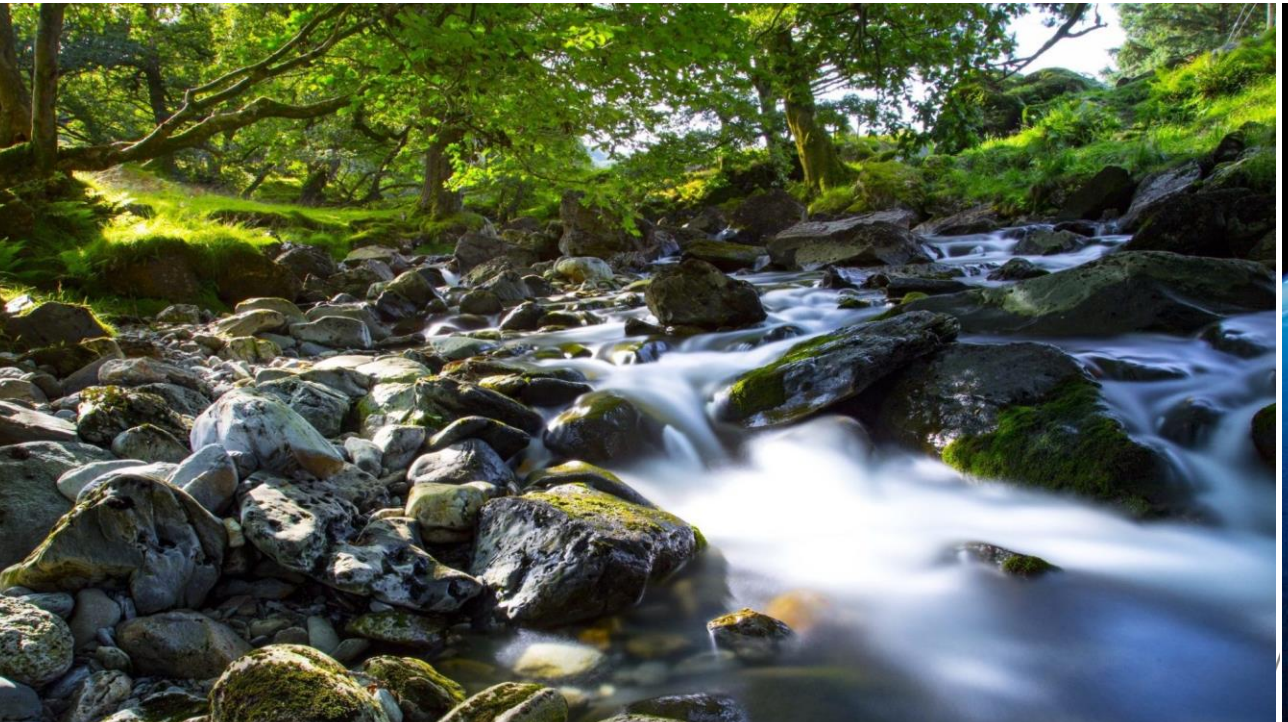
Positioning the next four lectures


Kafka
Horizontally scalable
Spark
Simple queries
Complex Queries
pub sub
C : A → B
MQTT
Vertically scalable
Esper

POLITECNICO
MILANO 1863

Positioning this lecture


Kafka
Horizontally scalable
Spark
Simple queries
Complex Queries
C : A → B
pub sub
MQTT
Vertically scalable
Esper



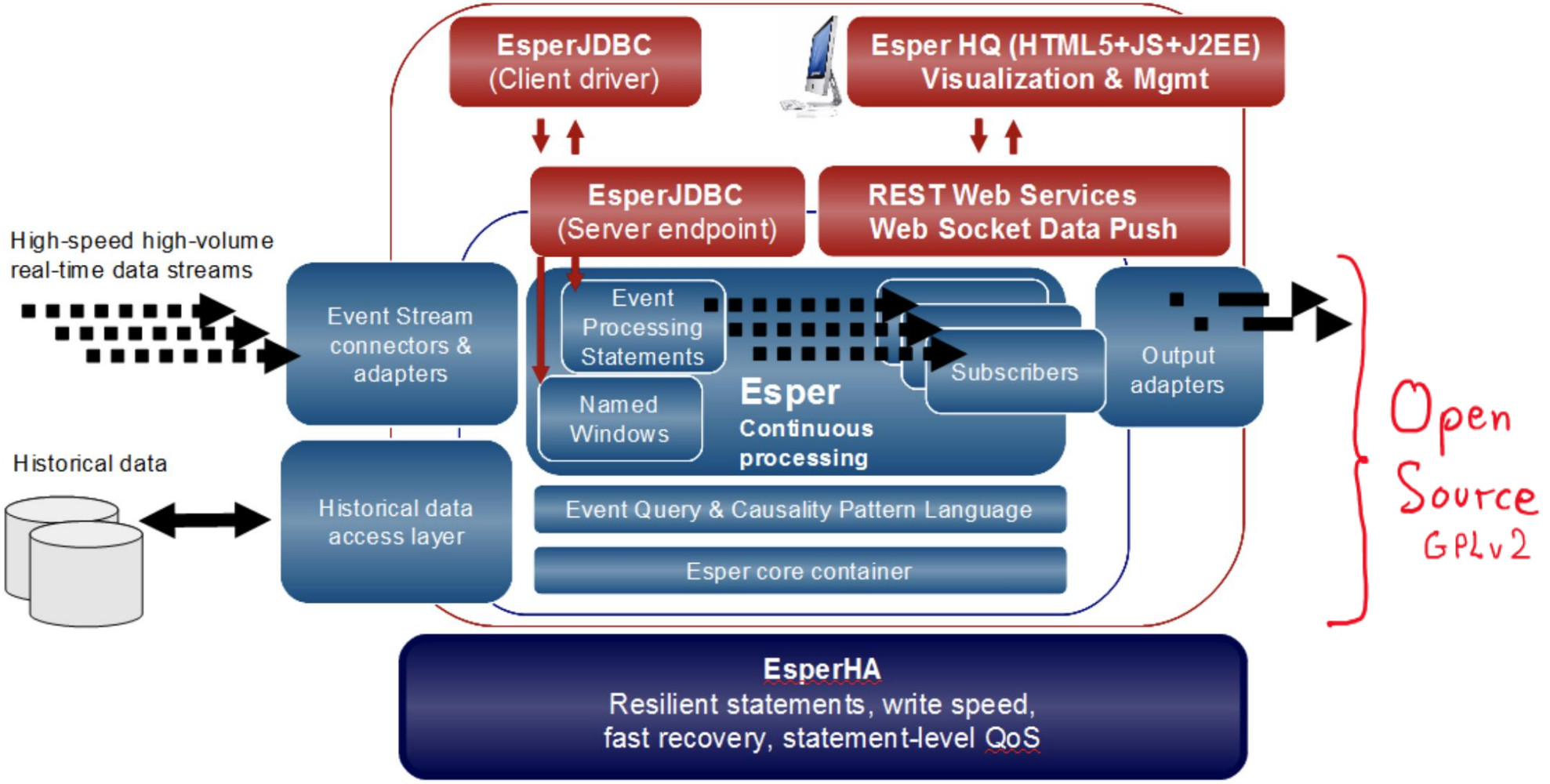
POLITECNICO
MILANO 1863

The Event Processing Language & its reference implementation Esper

What's Esper

- It is EsperTech's Complex Event Processing and Streaming Analytics software
- It turns large volume of disparate event series or streams into actionable intelligence.

Esper in a nutshell



[src: http://static.espertech.com/EsperTech%20technical%20datasheet.pdf]

POLITECNICO
MILANO 1863

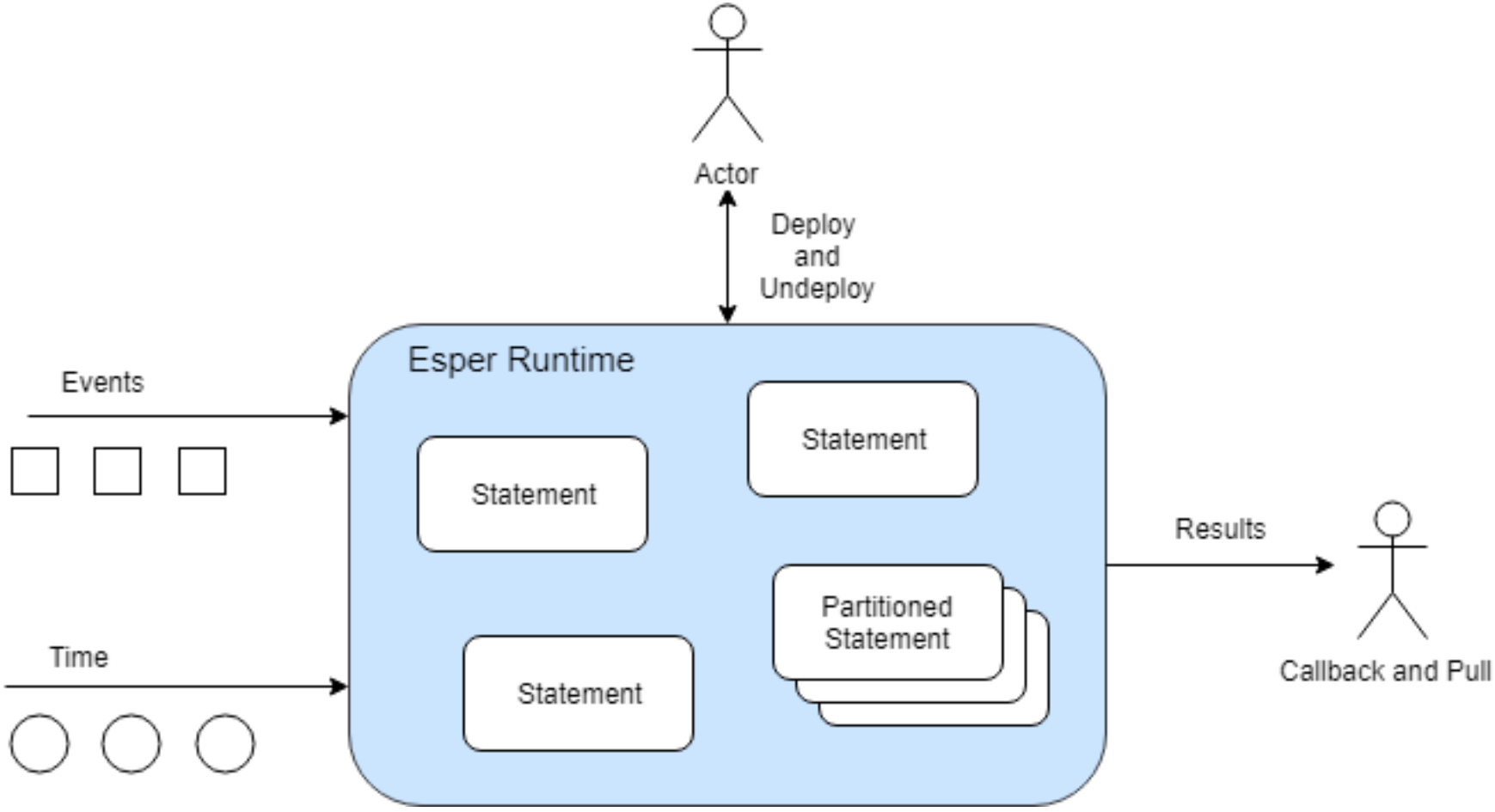
Features at a glance

- Implemented as a Java library
  - Can be embedded in any JVM application
- Designed for performance
  - High throughput
  - Low latency
- Interaction with static / historical data
- Configurable push or pull communication

EPL in a nutshell

- EPL: rich language to express rules (a.k.a., statements)
- As in the DSMS approach, it offers
  - Stream-to-relation operators, e.g., windowing
  - Relation-to-relation operators: selection, projection, join, aggregate, …
  - Relation-to-stream operators to produce output
- As in the CEP approach, it offers
  - comprehensive pattern detection
- Queries can be combined in a DAG

Esper & EPL



[src: http://esper.espertech.com/release-9.0.0/reference-esper/html/processingmodel.html]

The Basics of the Event Processing Language

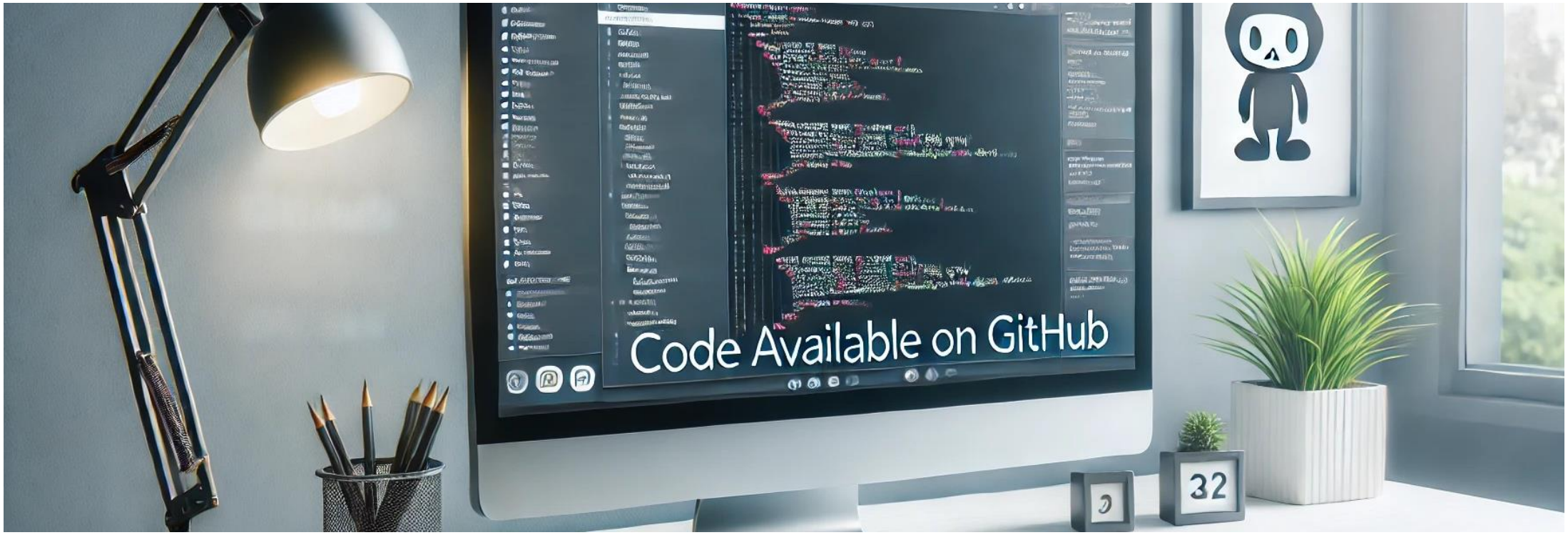
Running example

- Count the number of fires detected using a set of smoke and temperature sensors in the last 10 minutes
- Events
  - Smoke event: String sensor, boolean state
  - Temperature event: String sensor, double temperature
  - Fire event: String sensor, boolean smoke, double temperature
- Condition:
  - Fire: at the same sensor smoke followed by temperature>50 within 2 minutes

Code available on github!

- All the code described in this lecture is available at

  https://github.com/Streaming-Data-Analytics/Courseware/tree/main/Streaming%20Data%20Engineering/EPL/epl_firealarm



Declare event types

- Two ways
  - EPL *create schema clause*
  - Runtime configuration API *addEventType*
- Syntax

```
create schema
schema_name [as]
(property_name property_type
[,property_name property_type [,...])
[inherits inherited_event_type
[, inherited_event_type] [,...]]
```

Running example

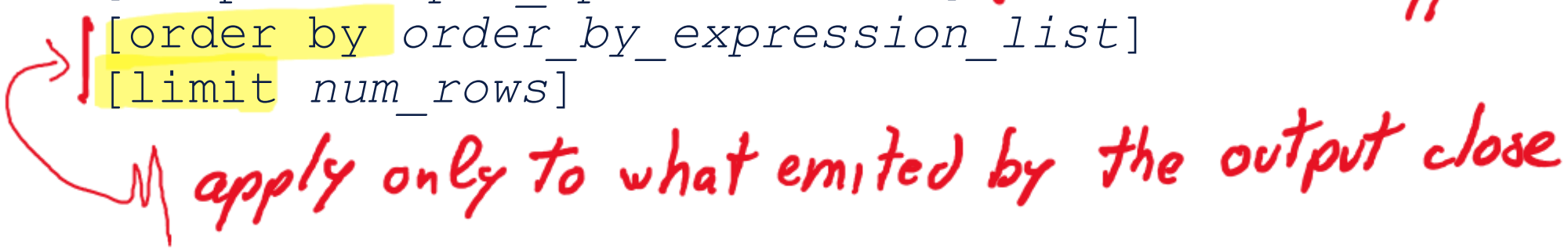
```
create schema SmokeSensorEvent(
      sensor string,
      smoke boolean
);

create schema TemperatureSensorEvent(
      sensor string,
      temperature double
);

create schema FireComplexEvent(
      sensor string,
      smoke boolean,
      temperature double
);
```

Event Processing Language (EPL)

- EPL is similar to SQL
  - Select, where, …
- Event streams and views instead of tables
  - Views define the data available for the query
  - Views can represent windows over streams
  - Views can also sort events, derive statistics from event attributes, group events, …

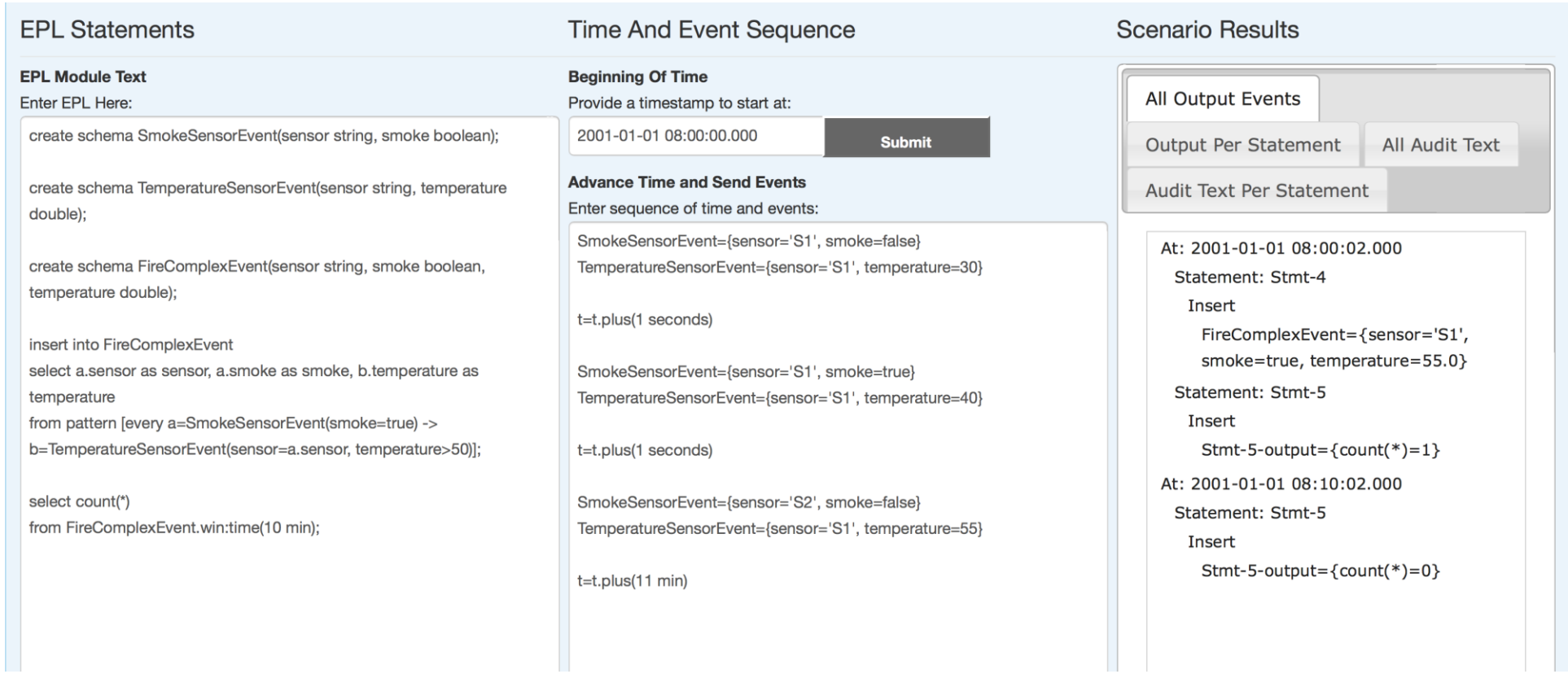
EPL syntax

```
[insert into insert_into_def]
select select_list
from stream_def [as name]
[, stream_def [as name]] [,...]
[where search_conditions]
[group by grouping_expression_list]
[having grouping_search_conditions]
[output output_specification]
[order by order_by_expression_list]
[limit num_rows]
```

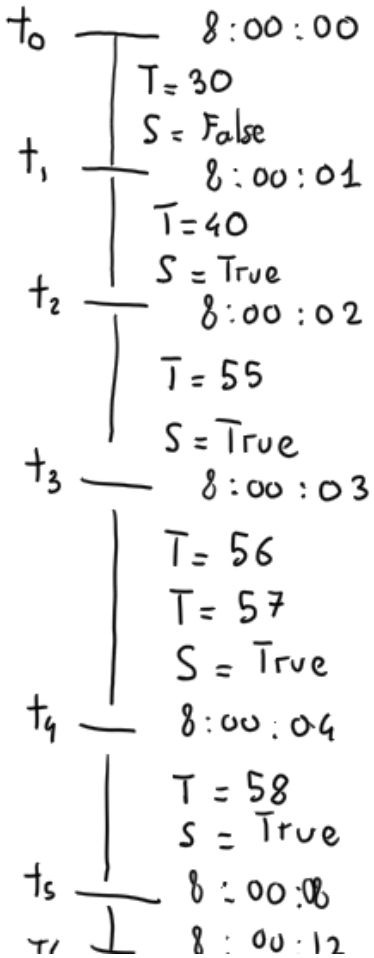


Running example

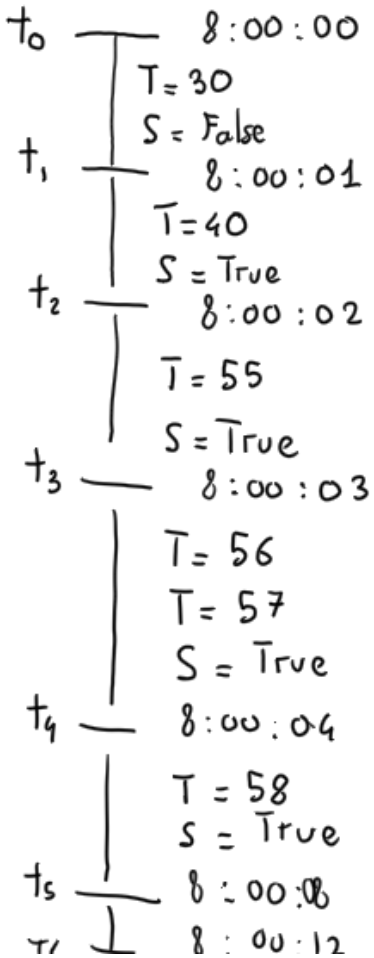
http://esper-epl-tryout.appspot.com/epltryout/mainform.html



Time line


t_0
8:00:00
T = 30
S = False
t_1
8:00:01
T = 40
S = True
t_2
8:00:02
T = 55
S = True
t_3
8:00:03
T = 56
T = 57
S = True
t_4
8:00:04
T = 58
S = True
t_5
8:00:08
8:00:12

Filtering



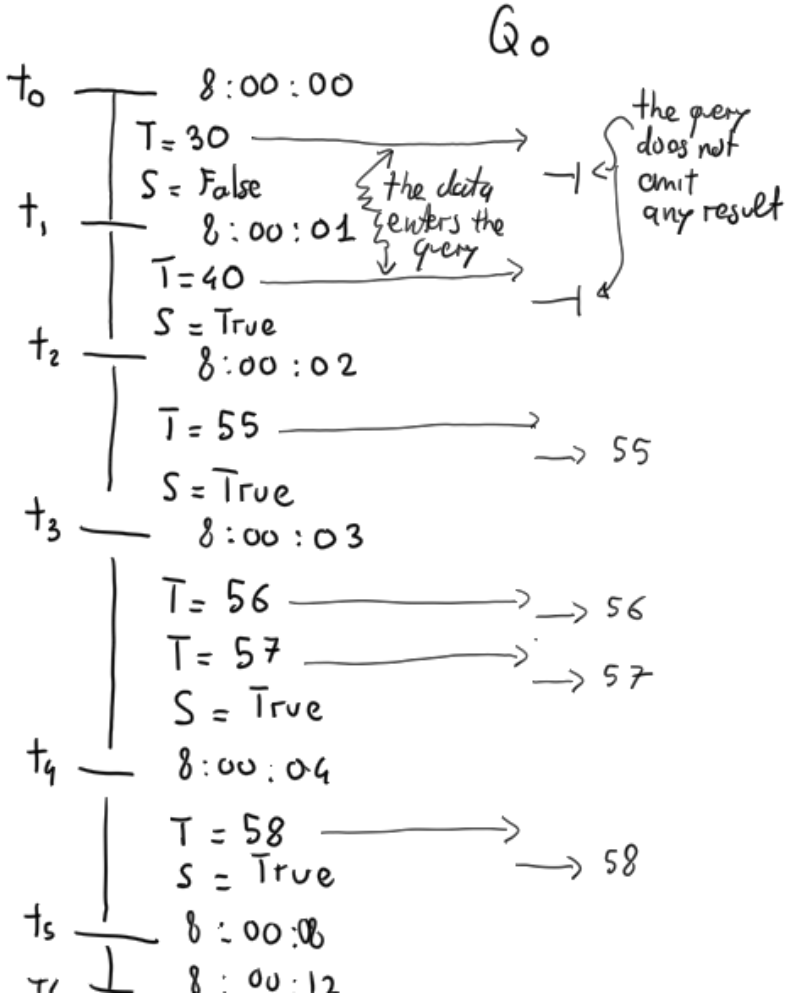
Q0 - the SQL-style

```
select *
from TemperatureSensorEvent
where temperature > 50;
```

Q0bis - the Event-Based System Style

```
select *
from TemperatureSensorEvent(temperature > 50) ;
```

Filtering the SQL style



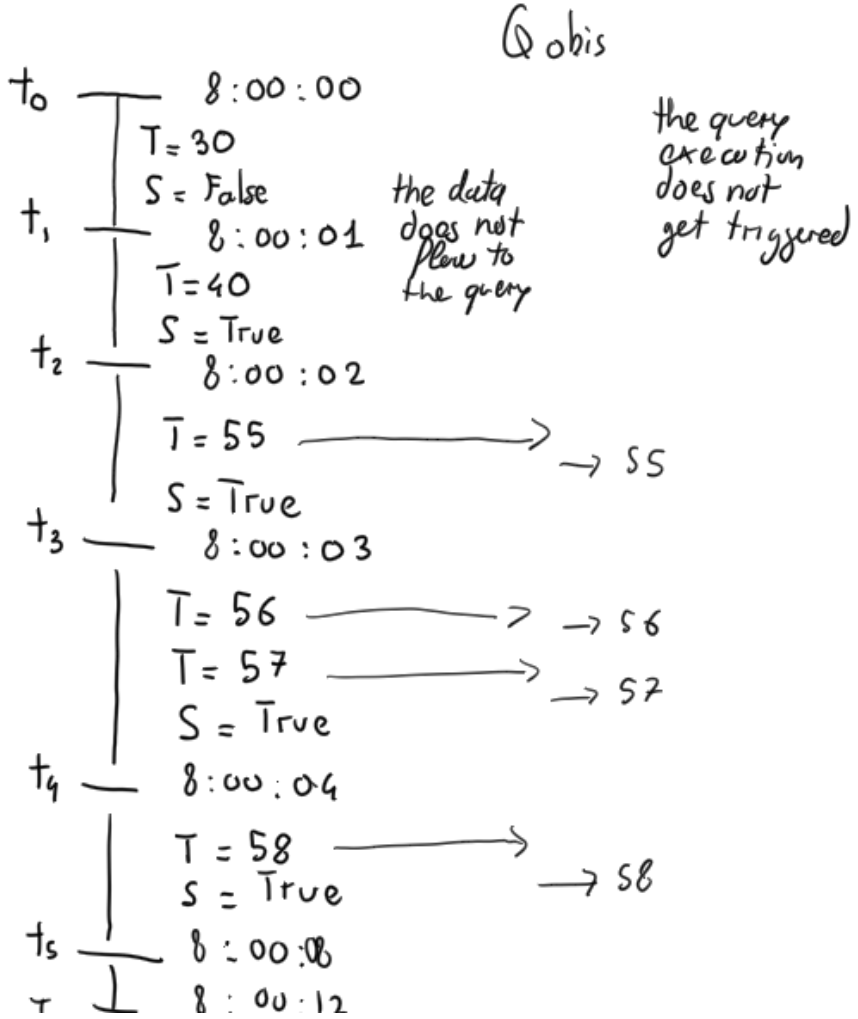
Q0 - the SQL-style

```
select *
from TemperatureSensorEvent
where temperature > 50;
```

Execution mode

Data points enter the query that filters them

Filtering the Event-Based System Style



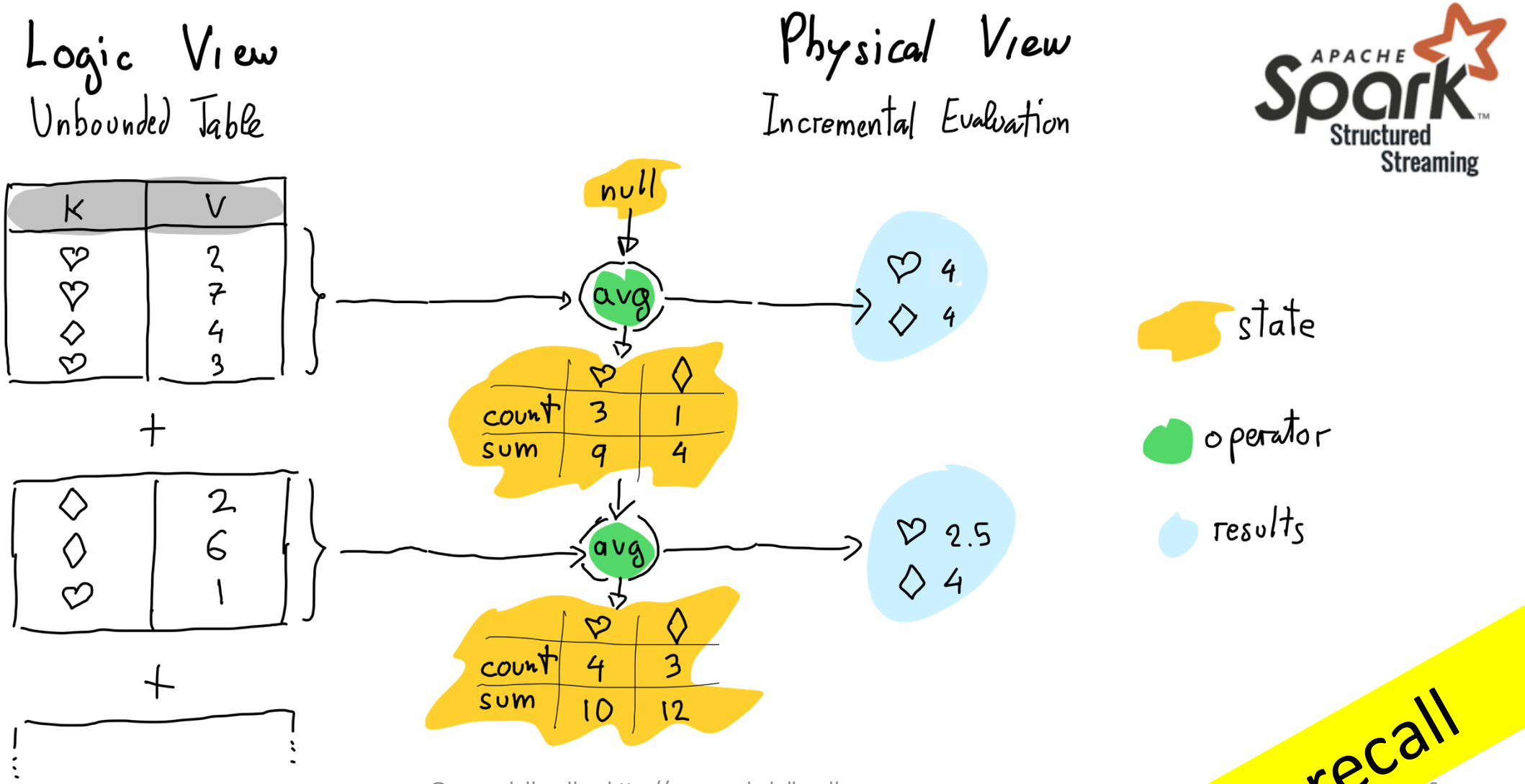
Q0bis - the Event-Based System Style

```
select *
from TemperatureSensorEvent(temperature > 50)
;
```

Execution mode

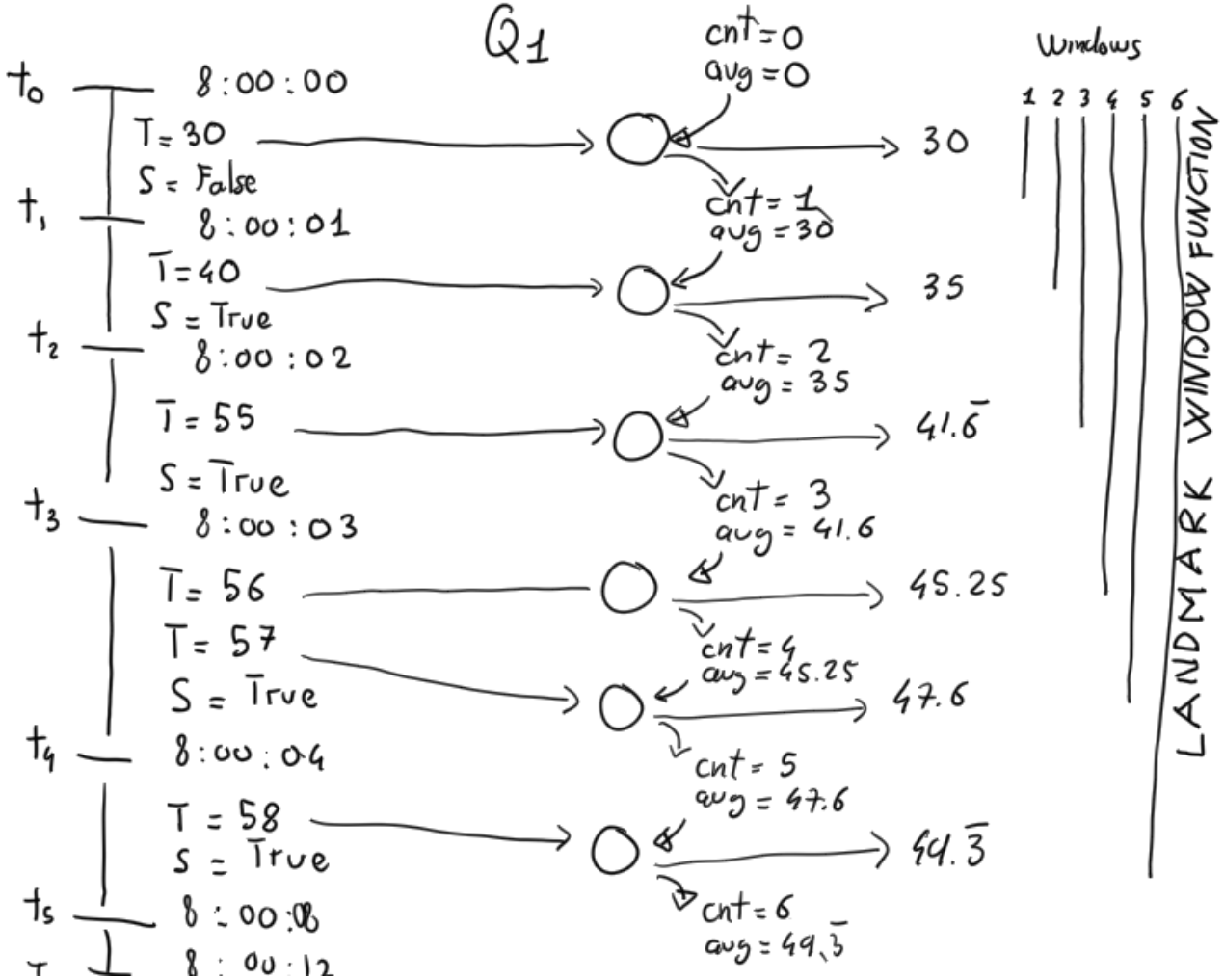
Data points are filtered before flowing into the query and the query execution does not get triggered

Modern DSMS



recall

Basic Aggregation




```
select sensor, avg(temperature)
from TemperatureSensorEvent
group by sensor;
```

Execution mode

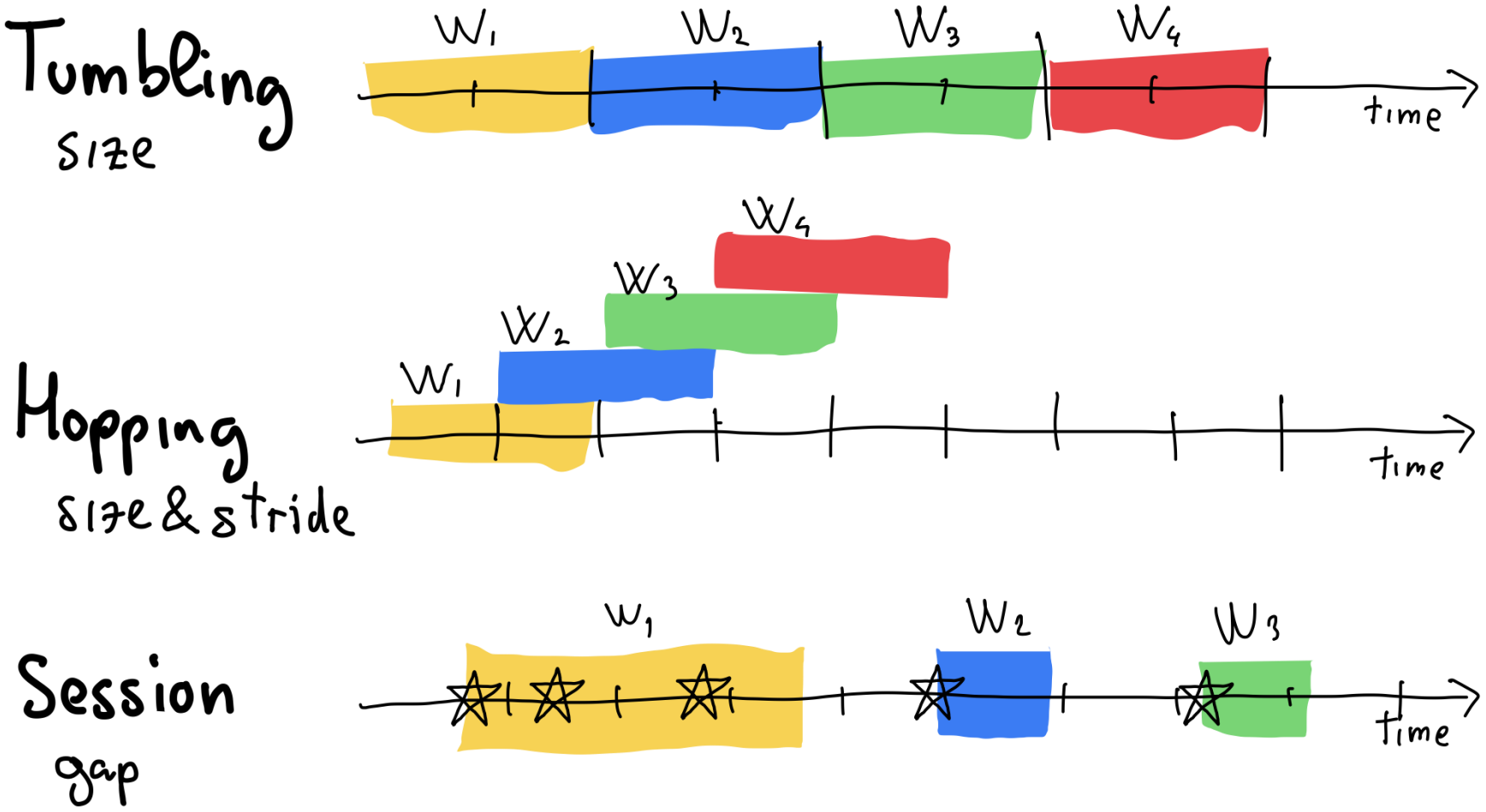
From a **logical** perspective, **data points cumulate in the landmark window** (a.k.a. *unbounded table*)**,** and the query emits a new avg for every data point

From a **physical** perspective, **the query is evaluated incrementally** and **maintains a state** that captures the number of events (cnt) seen so far and the previous avg. The new average only depends on the state and the new data point entering the query. Old data can be forgotten.

Data Windowing

- **Data windows** are for managing fine-grained event expiry.
- They instruct the engine how long to retain relevant events or under what conditions events can be discarded.
- Data windows operate on the level of individual queries, streams and subqueries.
- For example, use a time window to keep arriving events for N seconds. The engine let events go (expires) that are older than N seconds.

(The most important) Windows



recall

EPL basic window syntax

Type	Syntax	Description
Logical Sliding	win:time(*time_period*) #time(*time_period*)	Sliding window that covers the specified time interval into the past
Logical Tumbling	win:time_batch(*time_period* [, *reference point*] [, *flow control*]) #time_batch(*time_period …)*	Tumbling window that batches events and releases them every specified time interval, with flow control options
Physical Sliding	win:length(*size*) #length(*size*)	Sliding window that covers the specified number of elements into the past
Physical Tumbling	win:length_batch(*size*) #length_batch(*size*)	Tumbling window that batches events and releases them when a given minimum number of events has been collected

Logical (Q3) vs. Physical (Q4) Tumbling

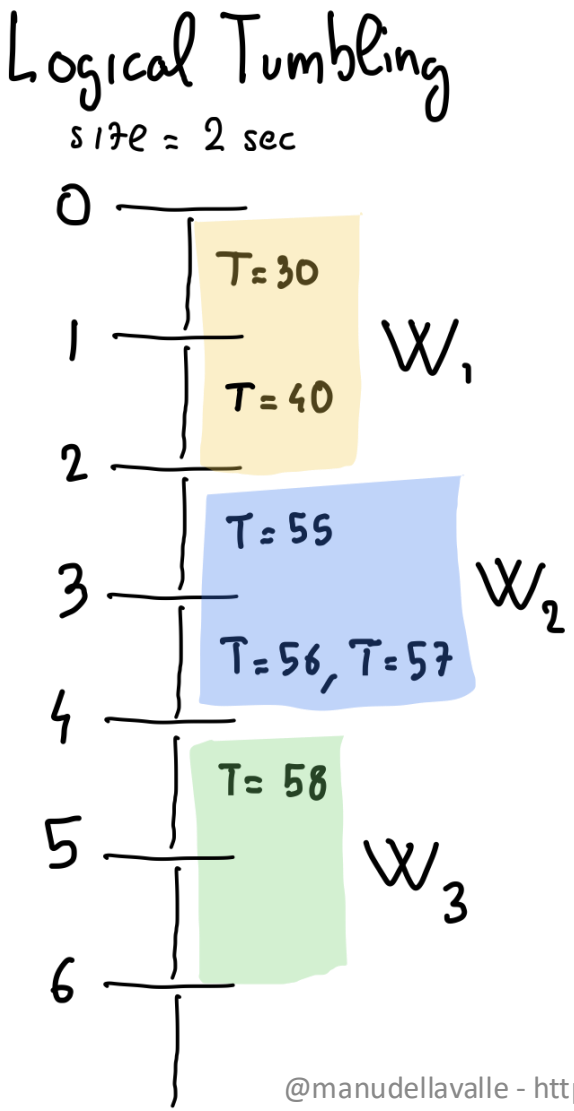
- The events received in the last 2 seconds every 2 seconds, a.k.a. logical tumbling window

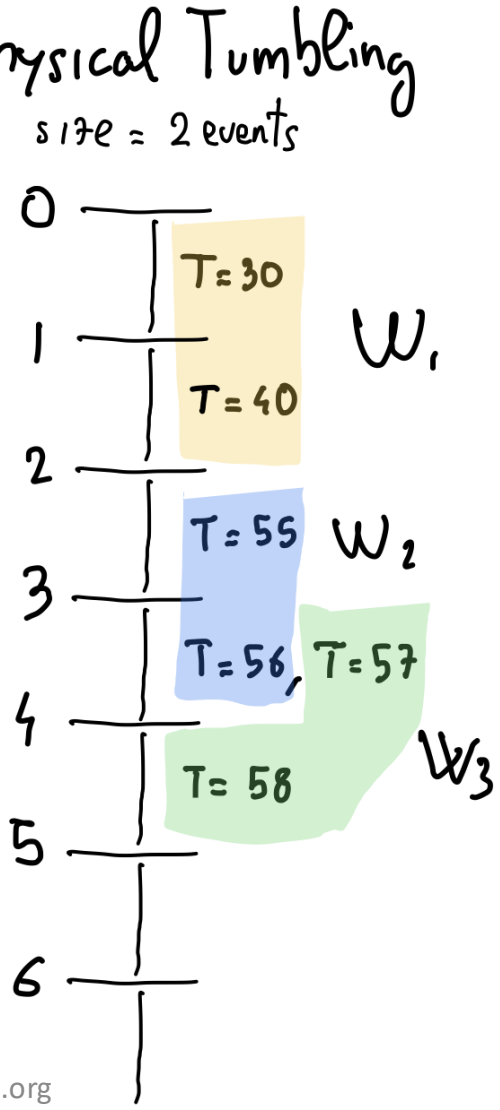
```
select *
from TemperatureSensorEvent.win:time_batch(2 seconds);
```

- The last 2 events received, a.k.a. physical tumbling window

```
select *
from TemperatureSensorEvent.win:length_batch(2);
```

Logical (Q3) vs. Physical (Q4) Tumbling





What's a sliding window?

Type	Syntax	Description
Logical Sliding	win:time(*time_period*) #time(*time_period*)	Sliding window that covers the specified time interval into the past
Logical Tumbling	win:time_batch(*time_period* [, *reference point*] [, *flow control*]) #time_batch(*time_period …)*	Tumbling window that batches events and releases them every specified time interval, with flow control options
Physical Sliding	win:length(*size*) #length(*size*)	Sliding window that covers the specified number of elements into the past
Physical Tumbling	win:length_batch(*size*) #length_batch(*size*)	Tumbling window that batches events and releases them when a given minimum number of events has been collected

Photo by Polesie Toys

Logical (Q5) vs. Physical (Q6) Sliding

- The events received in the last 4 seconds for every event, a.k.a. logical sliding window

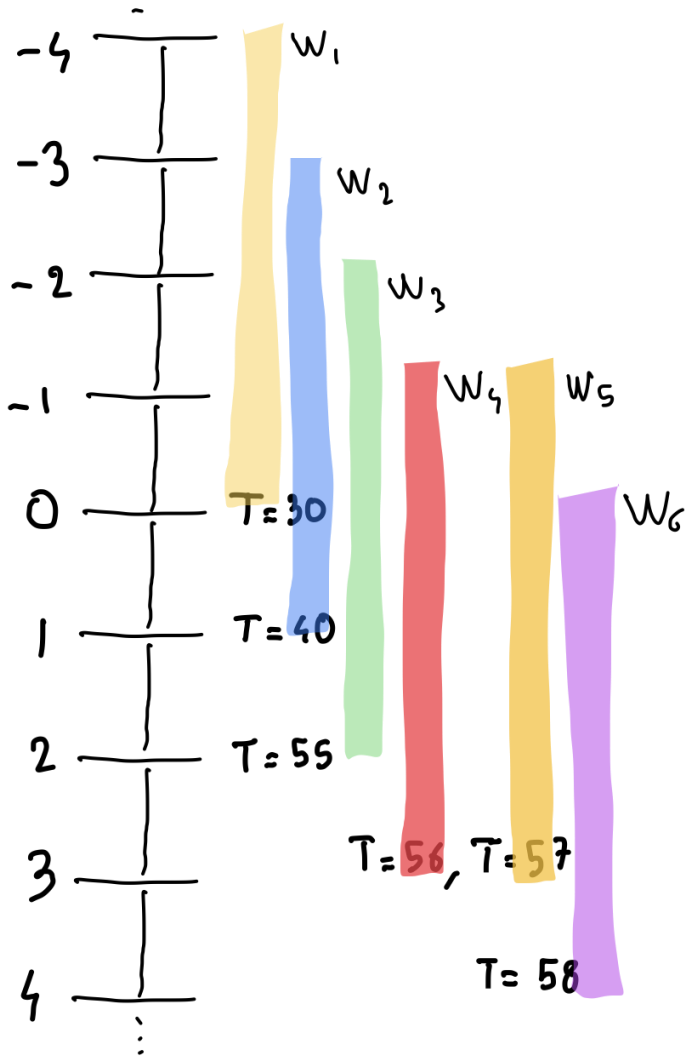
```
select *
from TemperatureSensorEvent.win:time(4 seconds);
```

- The last 4 events received for every event, a.k.a. physical sliding window

```
select *
from TemperatureSensorEvent.win:length(4);
```

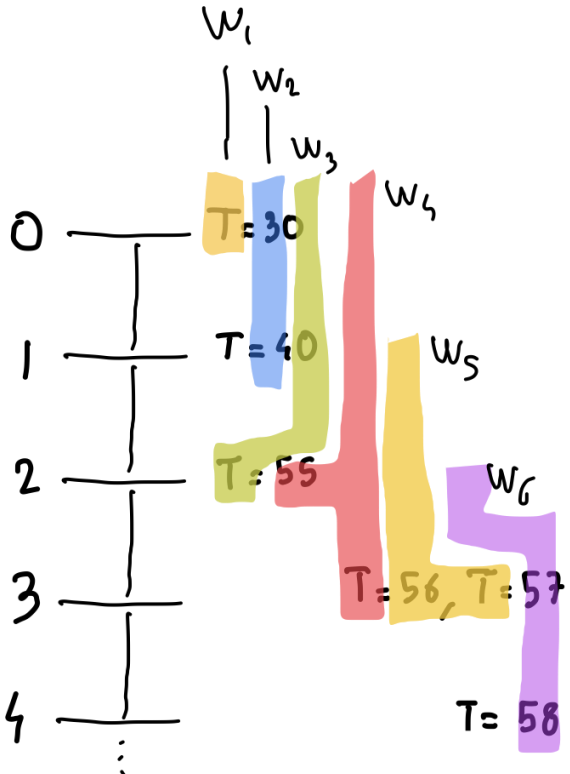
Logical Sliding (Q_5, Q_7)

size = 4 sec



Physical Sliding (Q_6, Q_8)

size = 4 events



:-O

Logical (Q7) vs. Physical (Q8) Sliding

- The average temperature observed by each sensor in the last 4 seconds for every event, a.k.a. logical sliding window

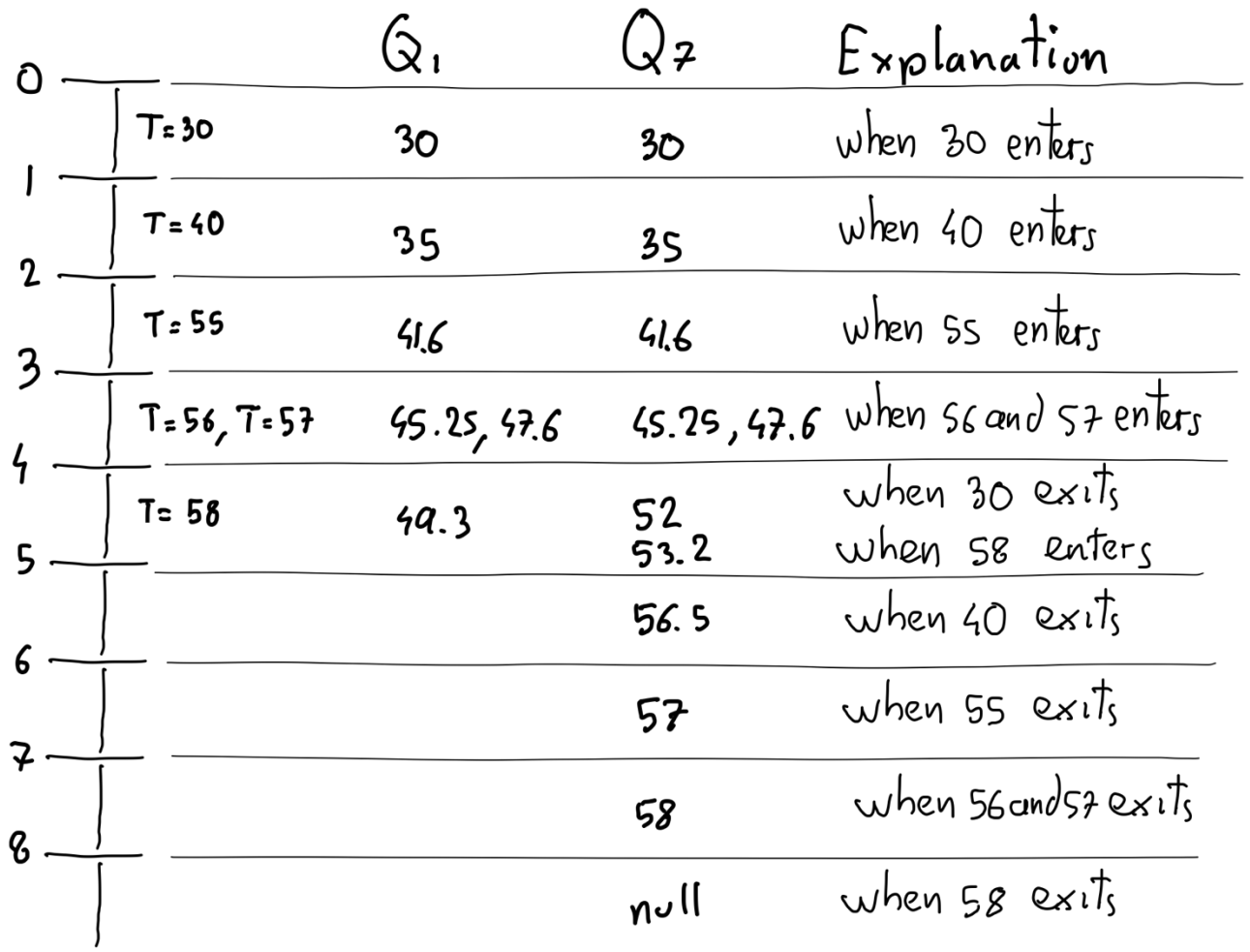
```
select sensor, avg(temperature)
from TemperatureSensorEvent.win:time(4 seconds)
group by sensor;
```

- The last 4 events received for every event, a.k.a. physical sliding window

```
select sensor, avg(temperature)
from TemperatureSensorEvent.win:length(4)
group by sensor;
```

On sliding window

- If you run Q5 and Q6, they appear to:
  - emit only the event they receive when they receive it
  - and, thus, to generate exactly the same results
- But this is not true, you can see if you run them as windowed aggregation query (see Q7 and Q8, respectively)



No Hopping Windows?

Aritan, S., 2012. Biomechanical measurement methods to analyze the mechanisms of Sport Injuries. *Sports Injuries: Prevention, Diagnosis, Treatment and Rehabilitation*, pp.19-26

EPL Output-control syntax

- The output clause

```
output      [all | first | last | snapshot]
every       output_rate [seconds | events]
```

  is used to stabilize and control output

- E.g., the average temperature of the last 4 seconds every 2 seconds, a.k.a. logical hopping window (Q11)

```
select sensor, avg(temperature)
from TemperatureSensorEvent.win:time(4 seconds)
group by sensor
output snapshot every 2 seconds;
```

EPL Output-control syntax
all | first | last | snapshot ?

- **first** ensures that the **first** result generated by the query is output immediately
- **last** outputs the **most recent** result at the time of the query's execution, discarding any intermediate results that may have been calculated
- **all** outputs **every** result generated by the query, including **both current and previous** calculations
- **snapshot** outputs **only** the **current state** of the query at the specified interval. Unlike all, it doesn't include previous results

:-O

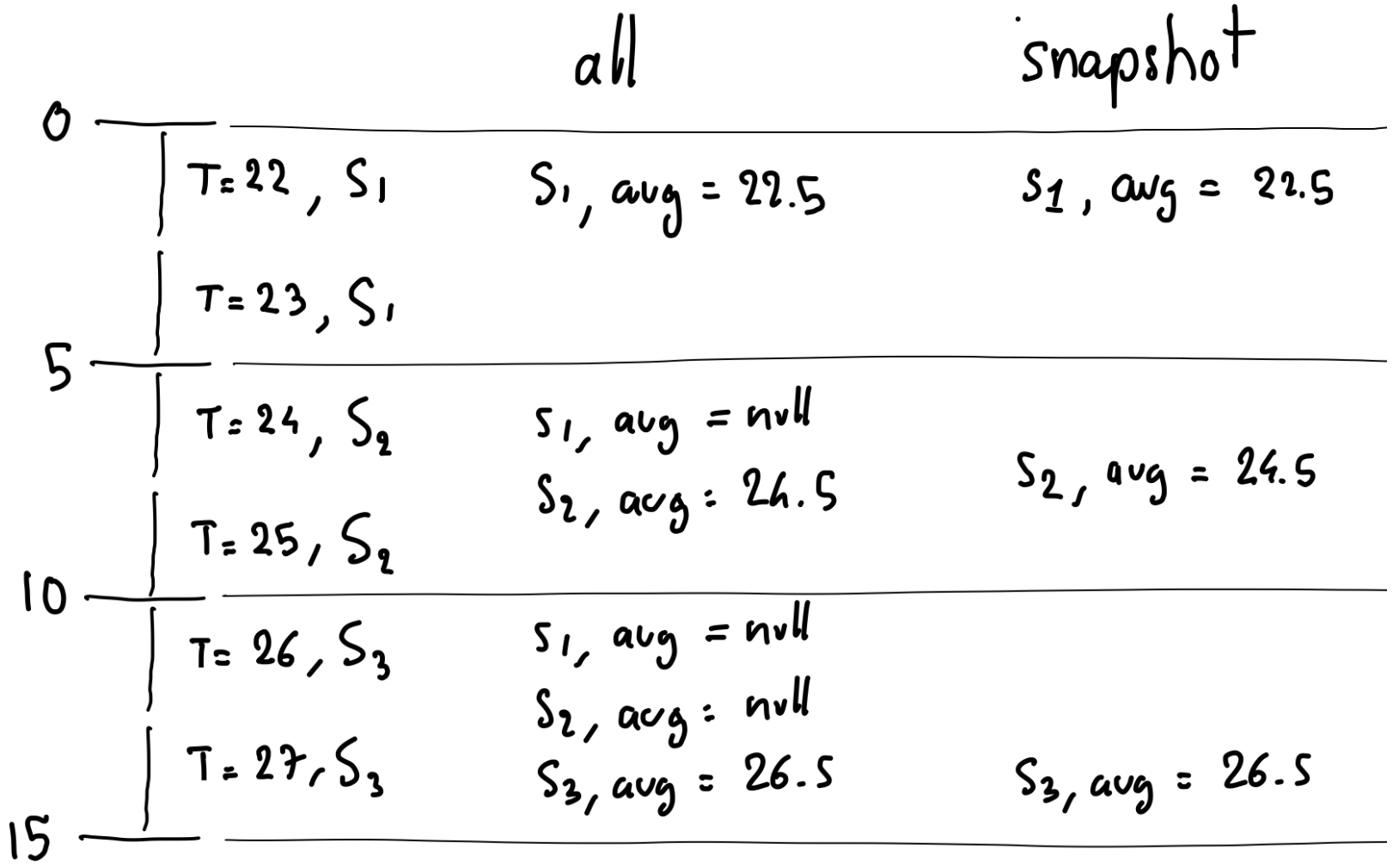
EPL Output-control syntax
all vs snapshot

POLITECNICO MILANO 1863

- Let's consider the following queries / events

```
select sensor, avg(temperature) as avgTemp
from TemperatureSensorEvent.win:time(10 seconds)
group by sensor
output all every 5 seconds;

select sensor, avg(temperature) as avgTemp
from TemperatureSensorEvent.win:time(10 seconds)
group by sensor
output snapshot every 5 seconds;
```

EPL Output-control syntax
all vs snapshot



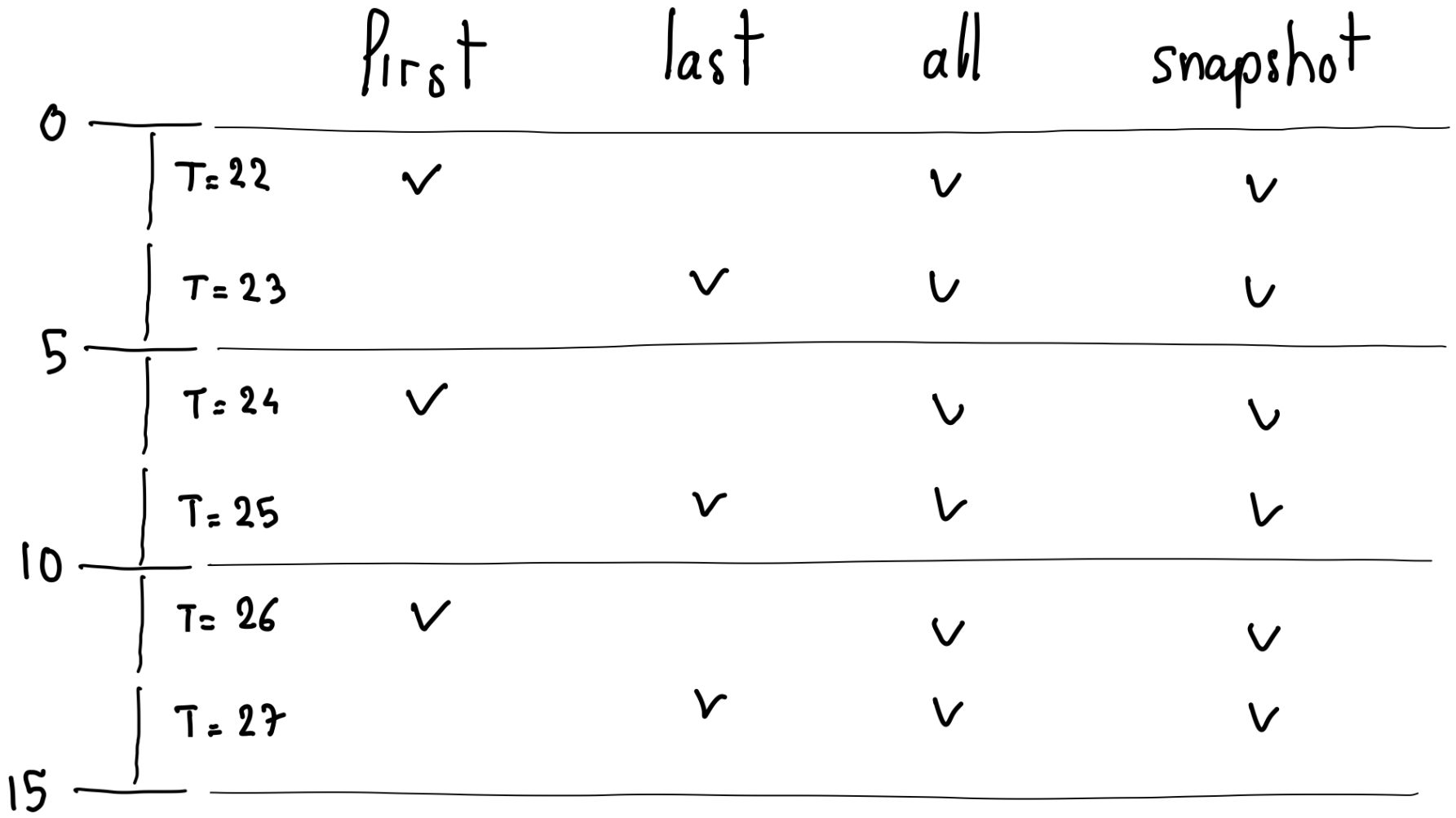
EPL Output-control syntax
first vs last vs all vs snapshot

POLITECNICO MILANO 1863

- Let's consider the following queries / events

```
select * from TemperatureSensorEvent.win:time(10 seconds)
output first every 5 seconds;
select * from TemperatureSensorEvent.win:time(10 seconds)
output last every 5 seconds;
select * from TemperatureSensorEvent.win:time(10 seconds)
output all every 5 seconds;
select * from TemperatureSensorEvent.win:time(10 seconds)
output snapshot every 5 seconds;
```

EPL Output-control syntax
first vs last vs all vs snapshot



Reference

- Esper documentation online
  - https://esper.espertech.com/release-9.0.0/reference-esper/html_single/
- Esper EPL online
  - http://esper-epl-tryout.appspot.com/epltryout/mainform.html

License and Acknowledgment

- This work is licensed under the Creative Commons Attribution-ShareAlike International Public License

- The original slides where prepared by Alessandro Margara for the PhD course on “Stream and Complex Event Processing in the Big Data Era” offered in 2019

http://streamreasoning.org/events/scep2019

EPL and Esper

The DSMS-like part of the language

POLITECNICO
MILANO 1863

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml